

chapter6

제6장 실시간 스트리밍 처리

5장 마지막에서 지연 이벤트라는 과제를 남겼다. 23:58에 발생한 민원이 00:03에 도착하면, 이미 닫힌 집계에 반영할 것인가 폐기할 것인가 — 그리고 그 판단을 누가, 어떤 규칙으로 내릴 것인가. 이 장은 그 규칙을 실제 스트리밍 엔진(Spark Structured Streaming) 위에서 구현하고 관찰한다. 먼저 “스트림을 무한히 자라는 테이블로 본다”는 Structured Streaming의 핵심 발상을 이해하고, 시간 창(window)으로 이벤트를 묶어 세는 tumbling-sliding 집계를 같은 데이터로 비교한다. 이어서 watermark를 선언해 지연 이벤트가 실제로 수용되는 경우(집계 갱신)와 폐기되는 경우(카운터 증가)를 각각 실측으로 확인하고, 그 위에 실시간 알림 조건을 설계한다. 실습(★★★☆☆)은 브로커 없이 파일 소스로 진행하므로 단독 실행이 가능하며, Java 17 이상이 필요하다.

학습 목표

Structured Streaming의 기본 구조(무한 테이블, micro-batch, trigger, output mode)를 자신의 말로 설명할 수 있다.

tumbling-sliding 윈도우 집계와 watermark를 실제 쿼리에 적용하고, 지연 이벤트의 수용·폐기를 검증할 수 있다.

실시간 공공 대시보드용 집계 쿼리와 알림 조건을 설계하고 그 한계를 설명할 수 있다.

6.1 스트리밍 DataFrame과 trigger 개념

학습 목표

“스트림 = 무한히 자라는 테이블”이라는 모델을 이해한다.

micro-batch-trigger-output mode가 각각 무엇을 결정하는지 구분한다.

직관: 배치 질의를 무한 테이블에 얹는다

2장에서 배치 처리를 “모아서 한 번에”, 스트리밍을 “도착하는 대로”라고 구분했다. 그런데 도착하는 대로 처리하는 코드를 처음부터 새로 배우는 것은 부담이 크다. Structured Streaming의 발상은 다르다: 스트림을 “행이 계속 추가되는, 끝이 없는 테이블”로 간주하고, 기존의 테이블 질의(DataFrame 연산)를 그 위에 그대로 적용한다.

민원 이벤트로 구체화하면 다음과 같다. 배치 세계에서 “지역구별 민원 건수”는 다음과 같다 — 테이블을 읽고, 지역구로 묶고, 계수한다. 스트리밍 세계에서 질의는 똑같다. 다른 것은 테이블이 “지금까지 도착한 이벤트 전부”이며 계속 자란다는 점뿐이다. 엔진은 새 행이 추가될 때마다 질의

결과를 **증분으로**(전체를 다시 계산하지 않고 바뀐 부분만) 갱신한다.

그림으로 정리하면 다음과 같다. 이벤트가 세 번에 걸쳐 도착할 때, “무한 테이블”과 “지역구별 건수” 결과 표는 이렇게 함께 자란다.

시점	무한 테이블(입력)에 쌓인 행	결과 표(지역구별 건수)
도착 1 후	강남구 민원 1건	강남구 1
도착 2 후	+ 마포구 1건, 강남구 1건	강남구 2, 마포구 1
도착 3 후	+ 강남구 1건	강남구 3, 마포구 1

핵심은 세 번째 도착에서 엔진이 “전체 4건을 처음부터 다시 계수하지” 않는다는 것이다. 이미 “강남구 2”라는 중간 상태를 유지하다가 새 이벤트 1건만 반영해 3으로 갱신한다. 이 중간 상태를 **상태(state)** 라고 부르며, 상태가 있기에 증분 계산이 가능하고, 상태가 축적되기에 나중에 “언제 폐기할 것인가”(watermark, 6.3)가 문제가 된다 — 이 장 전체의 복선이다.

이 모델의 힘은 학습 비용에 있다. 4장에서 Kafka Consumer를 다룰 때는 폴링 루프, 커밋 시점, 리밸런싱까지 직접 관리해야 했다. Structured Streaming에서는 “무엇을 세고 싶은가”(질의)만 선언하면, “어떻게 증분으로 계산하고 어디까지 처리했는지 기억하는가”는 엔진이 맡는다. 물론 비용이 없지는 않다 — 엔진이 대신 내리는 판단(특히 지연 이벤트 처리)을 **선언으로** 지시해야 하며, 그것이 6.3의 watermark다.

핵심 개념

스트리밍 DataFrame: 무한 테이블에 대한 참조다. `spark.readStream`으로 만들며, 소스는 Kafka·파일 디렉터리 등이다. 실습은 파일 소스를 사용한다 — 디렉터리에 새 파일이 도착하는 것을 “새 행 도착”으로 간주한다.

micro-batch: 엔진의 기본 실행 방식이다. 흐름을 아주 잘게 쪼갠 “작은 배치의 연속”으로 처리한다. 각 micro-batch는 그 시점까지 새로 도착한 데이터를 증분 계산한다. 4장의 폴링 루프(poll→처리→커밋)와 구조가 닮았다 — 다만 루프를 엔진이 돌린다. “한 건씩 처리해야 실시간이 아닌가”라는 의문이 제기될 수 있는데, micro-batch는 의도된 절충이다: 배치 단위로 처리하면 검증된 배치 실행 엔진을 재사용할 수 있고, “이 배치를 처리했는가”를 단위로 실패 복구가 단순해진다. 대신 최소 지연이 배치 주기만큼 생긴다 — 공공 대시보드 수준(초~분)에서는 문제가 되지 않는 절충이다.

trigger: micro-batch를 “언제” 돌릴지 정한다. 세 가지가 실무의 대부분을 덮는다.

기본(트리거 미지정): 앞 배치가 끝나는 대로 다음 배치를 실행한다 — 최저 지연이 목표일 때.

`processingTime="1 minute"`: 1분마다 실행한다 — 대시보드 갱신 주기처럼

“이만큼이면 충분”할 때. 주기를 늦추면 배치가 커져 효율이 향상되고 지연이 증가한다.

`availableNow`: 현재 축적된 데이터를 모두 처리하고 **종료**한다 — “한 시간에 한 번 실행되어 증분만 처리하는” 정기 작업. 스트리밍 코드로 배치 스케줄을 구현하는 셈이라 7장(배치)과의 접점이다.

output mode: 집계 결과 테이블에서 “무엇을” 내보낼지 정한다. 집계 결과는 새 이벤트가 올 때마다 갱신되는 표이므로, 매번 전체를 내보낼지(complete), 바뀐 행만 내보낼지(update), 다시는 안 바뀔 확정 행만 내보낼지(append)를 선택한다.

표 6.1 output mode 비교(집계 쿼리 기준)

모드	내보내는 것	직관	적합한 싱크(sink)
complete	결과 표 전체를 매번	“현황판을 전체를 다시 그린다”	작은 결과의 대시보드
update	이번 배치에서 바뀐 행만	“바뀐 칸만 고쳐 쓴다”	키 기반 저장소(Upsert, 5장)
append	다시는 안 바뀔 행만	“확정된 것만 내보낸다”	불변 로그·파일

update 모드와 5장의 연결에는 주목할 점이 있다. “바뀐 행만 내보낸다”는 것은 같은 키(창+지역구)의 행이 여러 번 내보내진다는 뜻이다 — 받는 쪽 저장소는 5장의 Upsert(키 기반 병합)로 받아야 일관성이 유지된다. append 모드는 “확정”이 필요하므로 watermark 없이는 집계에 쓸 수 없다 — 엔진이 “이 창은 다시는 안 바뀐다”고 판단할 근거가 watermark이기 때문이다(6.3).

이론이 정리한 네 가지 질문: what·where·when·how

trigger와 output mode가 각각 다른 것을 결정한다는 이 구분은 임의의 API 설계가 아니라, 스트리밍 처리 이론이 정리한 네 가지 질문에 대응한다. Akidau 외(2015)의 Dataflow 모델은 끝이 없는(unbounded) 데이터를 처리하는 모든 시스템이 답해야 할 질문을 넷으로 정리했다 — **무엇을 계산하는가(what), 어느 이벤트 시간 구간으로 묶는가(when), 언제 결과를 내보내는가(when), 갱신을 어떻게 반영하는가(how)**. 이 틀로 보면 앞서 학습한 개념들이 각각의 위치에 대응한다: 집계 질의가 what, 6.2의 윈도우가 where, trigger와 6.3의 watermark가 when(언제 계산하고 언제 창을 닫는가)에 해당한다. how는 “이전에 내보낸 결과와 이후의 갱신을 어떻게 관계 짓는가”(누적·정정)를 뜻하며, Spark의 output mode(전체·바뀐 행·확정 행)는 이 질문을 실용적으로 구현하는 한 방식이다 — 정확히 1:1 대응은 아니지만, 같은 물음에 답한다. 네 질문을 따로 답할 수 있다는 것이 스트리밍 설계의 핵심이다 — “10분 창을(where) 1분마다 갱신하되(when) 바뀐 행만 내보낸다(how)”처럼 조합이 독립적이기 때문이다. (적용 자격: Dataflow 모델은 스트리밍 일반을 대상으로 한 방법론·개념으로, 공공 부문의 직접 선례는 아니다. 이 책은 그 틀을 6장 실측으로 확인한다 — 2장에서 이미 같은 이론을 이벤트 시간·처리 시간 구분의 근거로 인용했다.)

민간에서 배워 공공으로: output mode에 책임성이 추가된다

output mode의 선택은 민간 실시간 시스템이 오래 다뤄 온 문제다. 게임 운영에서 “현재 동시접속자 수” 대시보드는 매 순간 전체 현황을 다시 그리므로 complete/update 모드이고, 같은 접속 스트림의 과금·정산 로그는 한번 확정되면 바뀌면 안 되므로 append 모드로 불변 파일에 남긴다. 민간에서 이 선택은 주로 성능·비용 문제다 — 대시보드가 잠정치임은 각주 한 줄로 충분하다. 공공에서는 같은 선택에 **책임성**이 추가된다. 어떤 수치가 “갱신되는 잠정치”이고 어떤

수치가 “확정 기록”인지를 산출물마다 명시하지 않으면, 감사에서 “대시보드와 보고서가 왜 다른가”라는 질문에 설계로 답할 수 없다. 바로 아래 공공 사례의 재난 상황판이 이 구분의 구체적 사례이며, 7장은 이 잠정치 대 확정치 구도를 정면으로 다룬다.

배치 질의와 무엇이 같고 무엇이 다른가

“배치 질의를 그대로 쓴다”는 말의 의미를 정확히 하면 다음과 같다. 같은 것은 **질의**를 쓰는 방식이고, 다른 것은 **실행과 책임의 구조**다.

항목	배치 질의	스트리밍 질의
질의 작성	DataFrame 연산	동일(DataFrame 연산)
읽기/쓰기	<code>read / write</code> — 한 번	<code>readStream / writeStream</code> — 계속
실행	호출 시 1회 계산 후 종료	<code>start()</code> 후 상주하며 증분 계산
중간 상태	없음(매번 전체 계산)	있음(집계 상태를 유지)
“어디까지 했나”	사람이 관리(재실행 범위)	엔진이 체크포인트(checkpoint)로 기록
실패 시	재실행으로 해결(7장, 단 맥등 필요)	체크포인트에서 이어서 재개 (보충)

마지막 두 행이 4장과 정확히 평행하다는 점을 확인해야 한다. 4장의 Consumer는 “어디까지 읽었나”를 커밋된 오프셋으로 기억했고, 스트리밍 쿼리는 같은 문제를 체크포인트로 푼다. 도구가 바뀌어도 “진행 위치의 기록”이라는 운영 문제는 그대로다.

실습에서 미리 보는 관찰 한 가지

실습 6.1의 실행 기록을 보면 micro-batch가 6개(0~5번) 실행되었는데, 홀수 번(1·3·5)은 내보낸 행이 0개다. 파일이 도착하면 그것을 처리하는 배치가 실행되고, 곧이어 추가 데이터를 확인하는 빈 배치가 한 번 더 실행되는 것이다. “micro-batch 번호 = 입력 파일 번호”가 아니라는 것, 즉 **엔진의 실행 단위와 데이터의 도착 단위는 다르다**는 것을 확인하는 소규모 실측이다.

공공 사례

재난 상황판을 예로 든다. 상황실 대시보드는 “지금까지의 지역별 신고 건수”를 계속 갱신해야 하는 반면(update 또는 complete), 사후 감사용 적재는 “확정된 10분 단위 집계”를 불변 파일로 남겨야 한다(append + watermark). 같은 스트림, 같은 질의라도 output mode가 다르면 산출물의 성격이 달라진다 — 요구사항(실시간 현황과 확정 기록)을 모드로 전환하는 것이 설계자의 역할이다.

검수 관점의 질문도 여기서 나온다. 납품 문서에 “실시간 집계를 제공한다”고 적혀 있다면, 검수자는 물어야 한다: trigger 주기는 얼마인가(지연의 하한), output mode는 무엇인가(수치가 갱신되는가 확정인가), 그리고 그 수치를 받는 저장소는 갱신을 감당하는 구조인가. 세 질문의 답이 없는

“실시간”은 명세가 아니라 수사(修辭)다. ### 핵심 정리 - Structured Streaming은 스트림을 무한 테이블로 보고, 배치 질의를 증분 실행한다. - trigger는 “언제 계산할지”, output mode는 “무엇을 내보낼지”를 정한다. - update 모드의 출력은 같은 키가 반복 도착하므로, 받는 쪽에 5장의 먹등 쓰기가 필요하다.

6.2 tumbling window와 sliding window

학습 목표

이벤트 시간 기준 창(window)이 왜 필요한지 이해한다.
tumbling과 sliding의 차이를 같은 실측 데이터로 비교한다.

직관: “지금까지 전부”가 아니라 “10분 단위로”

무한 테이블에서 지역구별 건수를 단순히 계수하면, 그 수는 시스템이 가동되는 한 계속 증가한다. 운영에서 필요한 것은 대개 “지금까지 전부”가 아니라 **“최근 10분에 몇 건”** 이다. 그래서 이벤트를 시간 창으로 잘라 창 안에서 계수한다. 이때 기준 시각은 도착 시각(처리 시간)이 아니라 **발생 시각(이벤트 시간)** 이어야 한다 — 지연 도착한 민원도 “발생한 10분”의 통계에 들어가야 보고가 정확하다(2장의 구분이 여기서 실전이 된다).

창을 만드는 방식이 두 가지다.

tumbling window(구르는 창): 시간축을 겹치지 않게 자른다. 22:00–22:10, 22:10–22:20, ... 벽시계의 10분 칸처럼 경계가 고정되어 겹치지 않는 구조다. 모든 이벤트는 정확히 한 창에 속한다.

sliding window(미끄러지는 창): 창 크기는 같지만 일정 간격으로 창을 민다. 크기 10분·슬라이드 5분이면 21:55–22:05, 22:00–22:10, 22:05–22:15, ... 창이 겹치므로 한 이벤트가 여러 창에 속한다(10분 창·5분 슬라이드면 정확히 2개 창).

한 이벤트가 어느 창에 속하는지는 수작업으로도 계산할 수 있다. 22:03에 발생한 이벤트를 예로 든다. tumbling(10분)에서는 시각을 10분 단위로 내림한 22:00이 창의 시작이므로 창은 22:00–22:10 하나다. sliding(10분 창·5분 슬라이드)에서는 “이 시각을 포함하는 10분짜리 구간 중 시작이 5분의 배수인 것”을 모두 찾는다 — 21:55–22:05와 22:00–22:10, 두 개다. 창 크기가 슬라이드의 정수배이면 창의 개수는 (창 크기 ÷ 슬라이드)개다: $10 \div 5 = 2$. 슬라이드를 2분으로 줄이면 한 이벤트가 5개 창에 속하고, 그만큼 결과 행과 엔진 상태도 늘어난다 — 촘촘한 감시에는 비용이 따른다.

창을 자르는 방식이 이 둘뿐인 것도 아니다. 민간의 클릭스트림(clickstream) 분석은 **세션 창(session window)** 을 사용한다 — 시간축을 고정 간격으로 자르지 않고, 한 이용자의 행동이 일정 시간 끊기면 거기서 창을 끊어 “세션당 페이지뷰 수”·“세션 체류 시간”을 계수한다. 창의 경계를 시계가 아니라 데이터의 공백이 정하는 방식이다. 공공에서도 한 민원의 접수부터 종결까지를 하나로 묶어

처리 소요를 확인할 때 같은 발상을 적용한다. 이 장은 tumbling과 sliding만 실측하지만, 창 방식의 선택지가 셋 이상이라는 것은 알아 두어야 한다 — “10분마다”라는 요구가 실은 “한 건의 처리 과정 단위로”인 경우가 있기 때문이다.

워크드 예제(worked example): 같은 이벤트, 두 가지 창(실측)

실습 6.1의 강남구 이벤트 7건(발생 시각 22:01, 22:03, 22:04, 22:08, 22:11, 22:14, 22:24)을 두 방식으로 계수하면 다음과 같다 — 아래 수치는 실제 실행 산출물이다(`ch5_window_counts.json1` , `ch5_sliding_demo.json`).

먼저 이벤트별 창 배정을 수작업으로 검산하면 다음과 같다. 위의 배정 규칙을 7건에 적용하면:

발생 시각	tumbling(10분) 창	sliding(10분·5분) 창 2개
22:01	22:00–22:10	21:55–22:05, 22:00–22:10
22:03	22:00–22:10	21:55–22:05, 22:00–22:10
22:04	22:00–22:10	21:55–22:05, 22:00–22:10
22:08	22:00–22:10	22:00–22:10, 22:05–22:15
22:11	22:10–22:20	22:05–22:15, 22:10–22:20
22:14	22:10–22:20	22:05–22:15, 22:10–22:20
22:24	22:20–22:30	22:15–22:25, 22:20–22:30

이 검산을 세로로 합치면 tumbling은 4·2·1, sliding은 3·4·3·2·1·1 — 그리고 엔진의 실제 출력이 정확히 이와 일치한다.

표 6.2 tumbling vs sliding 집계 비교(강남구, 실제 실행)

방식	창별 건수	건수 합
tumbling (10분)	22:00–22:10 → 4 , 22:10–22:20 → 2 , 22:20–22:30 → 1	7
sliding (10분 창·5분 슬라이드)	21:55–22:05 → 3, 22:00–22:10 → 4, 22:05–22:15 → 3, 22:10–22:20 → 2, 22:15–22:25 → 1, 22:20–22:30 → 1	14

합계에 주목할 필요가 있다. tumbling은 7(이벤트 수 그대로), sliding은 $14 = 7 \times 2$ 다 — 슬라이드가 창 크기의 절반이라 모든 이벤트가 정확히 두 창에 계수되었기 때문이다. 이것은 오류가 아니라 sliding의 정의다. 따라서 **sliding 창의 건수를 창끼리 더하면 이벤트 수가 아니다** — 보고서에 sliding 수치를 실을 때 반드시 유의해야 할 성질이다.

두 방식의 용도는 다르다. tumbling은 “10분마다 한 번, 그 10분의 확정 수치”를 제공하므로 정기 보고·확정 통계에 적합하다. sliding은 “5분마다, 최근 10분의 흐름”을 제공하므로 추세 감시·조기 경보에 적합하다 — 22:03에 이벤트가 집중되어도 tumbling은 22:10이 되어야 그 창이 완성되지만, sliding은 22:05 창(21:55–22:05)에서 이미 증가가 나타난다.

경계의 관례도 정리해 둔다. 창은 시작을 포함하고 끝을 포함하지 않는다 — 22:00–22:10 창은 [22:00:00, 22:10:00) 이다. 그래서 정확히 22:10:00에 발생한 이벤트는 앞 창이 아니라 다음 창(22:10–22:20)에 속한다. 사소해 보이지만, 수작업 검산과 엔진 결과가 1건 어긋나는 흔한 원인이 바로 이 경계 규칙이다. 집계 명세서에 경계 포함 여부를 명시하면 검수 시의 불필요한 논쟁을 줄인다.

핵심 코드

tumbling과 sliding의 코드 차이는 인자 하나다.

```
F.window("event_time", "10 minutes")           # tumbling: 슬라이드 없음
F.window("event_time", "10 minutes", "5 minutes") # sliding: 5분마다 새 창
events.groupBy(F.window("event_time", "10 minutes"), "district").count()
```

전체 코드는 [practice/chapter5/code/5-1-streaming-window-aggregation.py](#) 참고

민간에서 배워 공공으로: 광고 실시간 집계와 민원 현황판

tumbling과 sliding의 구분은 광고·커머스 실시간 집계에서 매일 쓰이는 실무 도구다. 광고 시스템은 “각 5분 구간에 노출·클릭이 몇 번”이라는 겹치지 않는 확정 수치로 광고주에게 정산하고(tumbling), 동시에 “최근 10분 클릭률 추이(1분마다 갱신)”로 이상 트래픽을 조기에 검출한다(sliding). 두 수치가 같은 클릭 스트림에서 나오지만 용도가 다르다 — 하나는 정산의 근거가 되는 확정치, 하나는 감시용 추이다. sliding 쪽에서 감시하는 지표가 클릭률(CTR, Click-Through Rate)과 전환율(CVR, Conversion Rate)이며, 값이 급락하면 광고 소재의 문제이거나 도착 페이지의 장애다 — 창 집계가 장애 감지 장치로 쓰이는 것이다. 이 기법이 공공으로 오면 대상만 바뀐다: 민원 유입 실시간 현황판은 “최근 1시간 지역별 신고 추이(10분마다)”를 sliding으로 제시하고, 월간 확정 통계는 “각 시간대 접수 건수”를 tumbling으로 산출한다. 민간에서는 sliding 합을 총량으로 오독해도 대개 사후 정정으로 끝나지만, 공공에서 같은 오독은 감사 지적이 된다 — 아래 공공 사례에서 해당 집계 오류를 실측 결과로 확인한다.

공공 사례

민원 월간 보고서는 “각 시간대에 몇 건”이라는 겹치지 않는 확정 수치를 요구한다 — tumbling이다. 반면 상황실의 “최근 1시간 신고 추이(10분마다 갱신)”는 sliding이다. 두 수치를 한 문서에 함께 제시할 때는 집계 방식의 차이를 명시해야 한다: 표 6.2의 실측처럼 같은 강남구 이벤트 7건이 sliding 표에서는 합계 14로 나타나므로, sliding 수치의 합을 총 건수처럼 해석하면 이벤트가 정확히 두 배로 과대 산출된다. 감사에서 “대시보드는 14건이라는데 보고서는 7건”이라는 질문을 받고 나서야 창 방식 차이를 설명하는 상황을 만들지 않으려면, 집계표에 창 방식·크기·슬라이드를 반드시 병기해야 한다. ### 핵심 정리 - 창 집계의 기준은 이벤트 시간이다 — 지연 도착해도 “발생한

시간대"에 계수해야 보고가 정확하다. - tumbling은 비중첩(확정 보고용), sliding은 중첩(추세 감시용)이며, sliding 건수의 합은 이벤트 수가 아니다. - 집계 산출물에는 창 방식·크기·슬라이드를 병기한다.

6.3 지연 이벤트와 watermark

학습 목표

watermark가 "언제까지 기다릴 것인가"의 선언임을 코드와 실측으로 이해한다.
지연 이벤트의 수용과 폐기를 각각 어떤 증거로 확인하는지 안다.

직관: 창구 마감 시간, 그리고 watermark의 정의

이벤트 시간 기준 집계에는 5장에서 본 근본 긴장이 있다: 창을 빨리 확정하면 늦게 도착한 이벤트를 놓치고, 무한정 기다리면 아무것도 확정하지 못한다(그리고 열린 창의 상태를 메모리에 계속 유지해야 한다). watermark는 이 긴장을 **선언 하나로** 타협한다.

은행 창구 마감 시간에 정확히 대응한다 — 창구는 지정된 마감 시각에 곧바로 닫지 않고 "마감 후 대기 중이던 고객까지는 처리한다"는 규칙만큼 더 열려 있다가 닫힌다. 마찬가지로 "22:00-22:10 창"은 22:10에 바로 닫히지 않고, "지연 10분까지 인정"이라는 규칙에 따라 열려 있다. 엔진은 지금까지 본 가장 최근 이벤트 시각에서 10분을 뺀 시각을 **watermark**로 유지한다 — "이보다 오래된 이벤트는 더 안 온다고 간주한다"는 기준선이다. 창의 끝이 이 기준선보다 오래되면 창은 닫히고 (상태 정리), 그 뒤에 온 지연 이벤트는 받지 않는다.

코드로는 선언 한 줄이다.

```
events.withWatermark("event_time", "10 minutes")
    .groupBy(F.window("event_time", "10 minutes"), "district").count()
```

여기서 반드시 이해해야 할 원리가 하나 있다. watermark는 **시계가 아니라 데이터에서 계산되며**, "지금 시각 - 10분"이 아니라 **"지금까지 본 이벤트 중 가장 최근 것의 이벤트 시각 - 10분"**이다. 그래서 데이터가 유입되지 않으면 watermark는 정지해 있고(창이 계속 열려 있고), 데이터가 대량 유입되면 watermark도 빠르게 전진한다(창이 빨리 닫힌다). 실습의 watermark 전진 과정을 규칙대로 따라가면 다음과 같다.

파일 처리 후	관측된 최대 이벤트 시각	watermark (= 최대 - 10분)	닫힌 창(끝 < watermark)
파일 1	22:09	21:59	없음
파일 2	22:18	22:08	22:00에 끝나는 창까지 (끝 < 22:08)
파일 3	22:24	22:14	22:10에 끝나는 창까지

파일 3 처리 후에는 창 22:00–22:10도 닫힌다(끝 22:10 < watermark 22:14). 즉 LATE_OK(22:04)가 만약 한 파일 늦게 도착했다면 판정은 수용에서 폐기로 바뀌었을 것이다 — 같은 이벤트, 같은 지연 허용값인데 스트림이 그 사이 얼마나 전진했는가에 따라 판정이 달라졌다.

워크드 예제: 수용과 폐기를 실측으로

실습 6.1은 지연 이벤트 두 건을 의도적으로 투입하고, 엔진의 판정을 관찰했다 (ch5_late_event_report.json). watermark의 계산 과정을 그대로 따라가면 다음과 같다.

수용된 지연 이벤트(LATE_OK) — 발생 22:04, 도착은 두 번째 파일(엔진이 22:09까지 관측한 뒤).

도착 시점의 watermark = 지금까지 본 최대 이벤트 시각(22:09) – 10분 = **21:59**.

이 이벤트가 속한 창은 22:00–22:10. 창의 끝(22:10)이 watermark(21:59)보다 미래다 → 창이 아직 열려 있다.

실측 결과: 창 22:00–22:10 강남구 건수가 **3 → 4로 갱신**됐다(update 모드가 바뀐 행을 다시 내보냄).

폐기된 지연 이벤트(LATE_DROP) — 발생 21:49, 도착은 세 번째 파일(엔진이 22:18까지 관측한 뒤).

도착 시점의 watermark = 22:18 – 10분 = **22:08**.

이 이벤트가 속한 창은 21:40–21:50. 창의 끝(21:50)이 watermark(22:08)보다 과거다 → 창은 이미 닫혔다.

실측 결과: 어떤 창도 갱신되지 않았고, 진행 메트릭의 numRowsDroppedByWatermark 가 **0 → 1로 증가**했다.

두 판정의 증거가 서로 다른 곳에 남는 점이 실무적으로 중요하다. 수용은 집계 결과(행 갱신)로 나타나지만, 폐기는 집계 결과 어디에도 나타나지 않는다 — 행이 없을 뿐이다. 그래서 폐기를 관찰하려면 집계 결과가 아니라 엔진 메트릭(numRowsDroppedByWatermark)을 지표로 감시해야 한다 (운영에서는 원천 입력 로그와의 대사로 보완할 수도 있다). 4장에서 “시나리오 B의 유실은 로그 없이 조용했다”고 했던 것과 같은 구조의 교훈이다: 유실된 데이터는 별도 기록 없이는 발견되지 않는다.

watermark는 폐기 규칙이자 청소 규칙이다

6.1에서 증분 계산이 상태(열려 있는 창들의 중간 집계)에 의존한다고 했다. watermark가 없으면 엔진은 “21:40 창에 지연 이벤트가 도착할 가능성 때문에” 모든 창의 상태를 무한정 유지해야 한다 — 하루만 가동해도 창×지역구 조합이 수천 개로 증가한다. watermark는 “이 창에는 더 도착하지 않는다”는 판단 기준이므로, 닫힌 창의 상태를 **폐기해도 되는** 시점이기도 하다. 즉 watermark를 길게 설정하는 것은 지연 수용을 늘리는 동시에 메모리에 상태를 오래 두는 결정이다(비용의 정량적 논의는 보충). 폐기 규칙과 청소 규칙이 동일한 기준이라는 것 — 이것이 “넉넉하게 하루로 설정하자”가 비용 없는 선택이 아닌 이유다.

watermark 값은 어떻게 정하는가

10분이라는 값은 임의의 값이 아니라 선택이다. 길게 설정하면 더 많은 지연을 수용하지만 확정이 늦고 상태(열린 창)가 많아진다. 짧게 설정하면 빠르지만 폐기가 증가한다. 올바른 순서는 **데이터의 실제 지연 분포를 관찰하고 정하는 것이다**.

지연 분포는 이벤트 발생 시각(send_ts)과 Consumer의 소비 시각 차이로 계산한다(4장 실측: 평균 3.5ms, 최대 110.7ms). 운영에서는 이벤트 시간과 도착 시간의 차이를 측정해 중앙값·상위 분위수·최댓값을 계산한다. 이 지연 분포를 기준으로 watermark를 정한다. 원천이 데이터를 30분 뒤에 게시한다면, 그 30분이 watermark 설정의 하한이 된다. 운영 중에는 폐기 건수도 기록하고, 폐기 건수가 늘면 지연 분포를 다시 계산한다.

하한을 원천이 정한다면 상한은 하류 판단의 시급성이 정한다. 실시간 광고 입찰(RTB, Real-Time Bidding)처럼 수십 밀리초 안에 결정을 내려야 하는 곳은 지연 허용을 0으로 설정하고 늦은 이벤트를 폐기한다 — 약간 부정확한 것보다 결정이 늦는 쪽이 비싸기 때문이다. 공공의 재난 센서도 시급성은 같지만, 버린 이벤트를 “반영하지 못한 분량”으로 남겨야 한다는 요구가 하나 더 붙는다. 그래서 폐기 카운터(numRowsDroppedByWatermark)는 도메인을 가리지 않고 운영 대시보드에 올리는 지표다 — 민간에서는 수집 지연이 증가했다는 신호로 해석하고, 공공에서는 거기에 더해 감사에 제출할 항목으로 활용한다.

현장의 운영 가이드도 이 트레이드오프를 같은 무게로 다룬다. Databricks의 watermark 실무 문서는 지연 허용값이 짧으면 유효한 지연 데이터를 잃고 길면 유지할 상태가 커진다는 균형을 설명한다 — 위에서 “실제 지연 분포에서 도출한다”고 한 원칙이 값을 정하는 구체적 방법인 이유다(적용 자격: Spark 벤더의 현장 가이드로, 판단 기준을 차용하되 분포 기반 도출은 이 책의 서술이다). 이 문서가 특히 설명하는 문제는 **여러 스트림을 join할 때의 watermark**다: 서로 지연 특성이 다른 두 소스를 합치면 엔진은 기본적으로 둘 중 느린 쪽(더 뒤쳐진 watermark)의 최솟값을 전체 기준으로 삼으므로, 빠른 소스의 창도 느린 소스만큼 늦게 닫힌다. 공공에서 민원(빠름)과 외부 센서(느림)를 한 파이프라인에서 합칠 때 실제로 발생하는 문제다.

이 개념은 Spark만의 것이 아니다 (Flink 대조)

watermark는 Spark의 발명품이 아니라 스트리밍 처리의 공통 개념이다. 다른 대표 엔진인 Apache Flink도 이벤트 시간과 watermark를 같은 의미로 사용한다 — Flink 문서는 watermark를 “이벤트 시간이 시각 T에 도달했으며 T보다 이른 이벤트는 더 오지 않을 것이라는 선언”으로 정의한다. 앞서

“관측된 최대 이벤트 시각 – 지연 허용”으로 계산한 것과 정확히 같은 발상이다. 차이는 제어의 세밀함에 있다: Spark Structured Streaming은 `withWatermark` 한 줄로 지연 허용만 선언하면 엔진이 나머지를 처리하는 반면(6.1의 “선언만 하면 엔진이 맡는다”), Flink는 watermark 생성 전략(주기적 생성·유휴 소스 처리 등)을 더 세밀히 지정할 수 있다. 이 대조가 학습에 주는 교훈은, watermark라는 개념을 이해하면 엔진이 바뀌어도 그대로 적용된다는 것이다 — 문법은 달라도 “언제까지 기다릴 것인가를 데이터에서 계산해 선언한다”는 원리는 같다. (적용 자격: Flink 공식 문서를 배경·타 엔진 대조로 인용하며, 공공 직접 선례는 아니다.)

민간에서 배워 공공으로: 실시간 매출 마감과 확정 문화

watermark의 “마감 시각” 발상은 민간 실시간 대시보드가 매일 부딪히는 문제다. 이커머스의 실시간 매출 대시보드는 결제 이벤트가 네트워크·재시도로 늦게 도착하므로, “이 시각 이후의 지연분은 다음 집계로 넘긴다”는 마감 규칙 없이는 수치가 계속 미세하게 변동한다. watermark는 그 변동을 명시적 규칙으로 전환한다 — “매출은 지연 N분까지 반영하고, 그보다 늦은 결제는 별도 카운터로 집계한다.” 공공으로 오면 대상이 매출에서 행정 수치로 바뀐다: 대기질 실시간 등급판은 늦게 도착한 측정값을 watermark 안에서만 반영하고, 재난 센서의 분 단위 창 집계도 같은 규칙을 따른다.

그런데 공공의 마감 문화가 이 결합을 특히 예민하게 만든다. 행정 보고는 “22시 대 민원 건수”처럼 시간대 확정 수치를 정시에 요구하고, 한번 보고한 수치의 소급 수정은 신뢰를 저해한다. 민간이라면 “실시간 매출은 잠정치”라고 각주 한 줄로 끝낼 문제가, 공공에서는 **잠정치와 확정치를 제도적으로 분리**하는 설계로 이어진다 — 상황판의 잠정 수치(watermark 안 근사치)와 마감 후 배치의 확정 수치(7장)를 서로 다른 산출물로 두고, 각각이 어떤 성격인지 명시한다. watermark는 이 분리의 경계선을 “며칠 뒤 사람이 판단”이 아니라 “데이터에서 계산된 규칙”으로 제공한다.

공공 사례

지자체 통계 담당자가 “22시 대 민원 건수”를 22:20에 확정해 보고했는데, 22:25에 지연 민원이 도착했다고 가정한다. watermark가 없다면 이 민원은 조용히 확정 수치와 어긋난 채 남거나, 소급 수정으로 보고 신뢰를 저해한다. watermark는 이 문제를 없애는 것이 아니라 **명시적 규칙으로 바꾼다**: “확정은 지연 10분까지 기다린 후이며, 그보다 늦은 건수는 폐기 카운터로 집계·보고한다.” 검수 관점에서 좋은 시스템은 지연을 완벽히 처리하는 시스템이 아니라, 지연 처리 규칙이 문서화되고 그 예외(폐기)가 관측되는 시스템이다. ### 핵심 정리 - watermark = (관측된 최대 이벤트 시각 – 지연 허용)이며, 창이 끝이 이보다 오래되면 창을 닫고 지연분을 폐기한다. - 수용은 집계 갱신으로 나타나지만, 폐기는 집계 결과에 흔적이 없어 메트릭 (numRowsDroppedByWatermark)으로 관찰한다. - 지연 허용값은 실제 지연 분포에서 도출하고, 폐기 카운터를 운영 지표로 감시한다.

6.4 실시간 알림 조건 설계

학습 목표

집계 스트림 위에 임계 기반 알림을 설계한다.
“미확정 창 알림”의 주의점과 알림 중복 문제를 이해한다.

핵심 개념

윈도우 집계가 만들어지면 알림은 그 위의 필터다: “10분 창에서 같은 지역구 민원이 N건 이상이면 알린다.” 그러나 단순히 보이는 이 규칙에는 지금까지 다른 문제가 모두 다시 등장한다.

미확정 창의 알림: update 모드에서 창의 수치는 지연 이벤트로 계속 증가한다. 강남구 창 22:00–22:10은 처음에 3이었다가 지연분 수용으로 4가 되었다(6.3 실측). 임계가 4라면 — 알림은 창이 닫히기 전, 두 번째 갱신에서야 발화한다. 반대로 “3에서 알림을 안 보냈는데 나중에 4가 됨”을 놓치지 않으려면 갱신마다 조건을 재평가해야 한다. **빠른 알림(미확정 수치 기준)과 정확한 알림(확정 수치 기준)은 다른 설계다.**

알림 중복: 갱신마다 조건을 재평가하면 같은 창이 조건을 두 번 만족할 수 있다(4에서 알림, 5에서 또 알림). 알림 채널에 같은 경보가 반복 도착하는 것 — 4.5장의 중복 문제가 알림 계층에서 재현된 것이다. 해법도 같다: 알림의 중복 제거 키(예: 창 시작 시각+지역구+규칙 ID)를 정하고, 발송 기록에 유니크 제약을 건다(5장).

무입력 상태 감지의 한계: 알림 조건은 “데이터가 있는데 많음”만 검출한다. “데이터가 전혀 유입되지 않음”(수집 장애)은 어떤 창도 임계를 넘지 않으므로 감지되지 않는다. 더 까다로운 점은, 집계 스트림에서 “0건인 창”은 건수가 0인 행으로 나오는 것이 아니라 **행 자체가 없다는 것**이다 — 이벤트가 없으면 창도 만들어지지 않는다. 따라서 “0건이면 경보”라는 조건은 집계 결과 위에서 적용할 수 없고, “일정 시간 동안 결과 행이 하나도 안 나왔음”을 감시하는 별도 장치(하트비트(heartbeat)·유입량 하한 감시)가 필요하다 — 11장 모니터링에서 확장한다. 4장 시나리오 B의 유실, 6.3의 폐기에 이어 세 번째로 확인하는 같은 교훈이다: **유입되지 않은 데이터는 별도 장치 없이는 감지되지 않는다.**

워크드 예제: 실습 데이터로 알림을 따라가기

임계를 “10분 창에서 같은 지역구 4건 이상”으로 정하고, 실습 6.1의 실제 갱신 흐름에 대응시켜 정리하면 다음과 같다.

파일 1 처리: 창 22:00–22:10 강남구 = 3. 임계(4) 미달 — 알림 없음.

파일 2 처리: 지연 이벤트 수용으로 같은 창이 3 → 4로 갱신. **이제 임계 도달 — 그런데 이 시점은 창이 끝난 지(22:10) 한참 뒤다.**

만약 “창 확정 후 평가”를 택했다면: 이 창은 파일 3 처리 후에야 닫히므로(watermark 22:14 > 창 끝 22:10) 알림은 더 늦어진다.

이 짧은 흐름에 알림 설계의 세 선택지가 모두 포함되어 있다. ① 갱신마다 평가하면 4가 되는 순간 알린다(빠르지만, 3에서 “아직 아님”을 판단했던 기록과 함께 관리해야 한다). ② 확정 후 평가하면 지연 이벤트 때문에 판정이 뒤집히는 오경보는 없지만 창 크기 + watermark만큼 늦는다(임계값 자체가 부적절해 발생하는 오경보는 확정 후에도 남는다 — 이는 평가 시점이 아니라 임계 설계의 문제다). ③ 그리고 어떤 선택이든, 이후 같은 창이 5로 또 갱신되면 재발화를 막을 중복 제거 키가 필요하다.

표 6.3 알림 조건 설계표(예시 — 값은 예시 설계값)

항목	예시	설계 질문
대상 집계	10분 tumbling, 지역구별 건수	창 방식·크기는 목적에 맞는가
임계	동일 지역구 ≥ 10 건	평시 분포 대비 몇 배인가
평가 시점	갱신마다(빠름) / 창 확정 후(정확)	오경보와 지연 중 무엇이 비싼가
중복 제거 키	창 시작+지역구+규칙 ID	같은 경보의 재발송을 무엇으로 막는가
침묵 감시	하트비트 없음·유입량 하한 감시	“유입 중단 장애”는 무엇이 검출하는가

이 표는 알림 하나를 추가할 때마다 채워야 할 최소 문서다. 특히 “설계 질문” 열은 검수·감사 담당자가 알림 체계를 평가할 때 그대로 질문 목록이 된다 — 임계의 근거가 평시 분포 관찰인가 임의 숫자인가, 오경보·늦은 경보 중 무엇을 감수하기로 했고 그 결정을 누가 승인했는가.

민간에서 배워 공공으로: 급증 알림과 형평성 제약

실시간 임계 알림은 민간 운영의 표준 장치다. 게임 운영은 동시접속 급증(이벤트 출시·버그)을 창 집계 위의 임계로 검출해 서버 증설을 트리거하고, 결제 시스템은 이상 트래픽(짧은 시간 같은 카드의 반복 결제)을 실시간으로 검출해 차단한다. 이 차단 판단의 입력도 창 집계다 — 1분 sliding 창으로 “같은 카드 번호의 직전 거래 금액 합”을 뽑아 사기 점수(fraud score)를 매기고, 점수가 임계를 넘으면 거래를 막거나 추가 인증을 요구한다. 임계를 정하는 계산은 정밀도(precision)와 재현율(recall)의 비용 비교다: 낮게 설정하면 사기를 더 검출하는 대신 정상 거래를 차단하고, 높게 설정하면 반대가 된다. 이 기법을 공공으로 옮기면 대상이 재난·민원 급증 감지가 된다 — “10분 창에서 같은 지역구 신고가 N건 이상이면 상황실에 경보.” 여기까지는 광고 이상 트래픽 감시와 구조가 같다.

그러나 공공의 알림은 두 지점에서 변형된다. 첫째는 **형평성**이다. 민간은 매출이 큰 지역·고객을 우선 감시해도 문제가 없지만, 공공의 급증 임계를 지역 인구·평시 민원량으로 보정하지 않으면 대도시만 늘 경보가 뜨고 소도시의 상대적 급증은 묻힌다 — 임계를 절대 건수가 아니라 “평시 분포 대비 몇 배”로 설정해야 하는 이유다(표 6.3의 설계 질문). 둘째는 **책임성**이다. 민간의 오경보는 대개 비용 낭비로 끝나지만, 공공의 대피 경보 오발령은 시민 신뢰를 저해한다 — 그래서 “빠른 알림(미확정 수치)”과 “정확한 알림(확정 수치)” 중 무엇을 택했는지, 그 결정을 누가 승인했는지가 알림마다 문서로 남아야 한다. 민간이 A/B 테스트로 알림 임계를 최적화하는 자리에, 공공은 이 승인·문서화 절차를 둔다.

공공 사례

재난 초기 대응에서 오경보(false alarm)의 비용과 늦은 경보의 비용은 비대칭이다 — 대피 문자를 잘못 보내는 비용보다 못 보내는 비용이 크다면, 평가 시점은 “갱신마다(빠름)”로 두고 오경보를 감수한다. 반대로 언론 브리핑용 수치는 확정 후에만 산출한다. 같은 스트림에서 두 알림 정책이 공존할 수 있으며, 중요한 것은 각 알림이 어느 정책(빠름/정확)인지 문서화되어 수신자가 그 성격을 아는 것이다.

역방향: 이 장의 설계를 민간 현장에 옮기면

지금까지는 민간 패턴을 공공으로 옮겼다. 반대 방향도 성립한다 — 이 장에서 만든 창 집계·watermark·알림 설계를 그대로 민간 현장에 적용하면 무엇이 유지되고 무엇이 바뀌는지 네 현장에서 확인한다.

실시간 추천의 피쳐 집계에서는 이용자가 최근 10분간 조회한 상품 범주를 sliding 창으로 계수해 추천 모델의 입력으로 사용한다. 출력 구조가 실습 6.1과 같다 — update 모드로 바뀐 행만 내보내고, 같은 이용자·같은 창의 행이 여러 번 나오므로 받는 쪽 피쳐 스토어(8장)가 키 기반 Upsert로 받는다(5장). 바뀌는 것은 창 크기를 정하는 근거다. 공공 현황판이 “보고 주기가 10분”이라 10분 창을 설정한다면, 추천은 “이용자가 관심을 유지하는 시간”으로 설정하고 응답 속도를 정확도보다 앞에 둔다.

사기 탐지 파이프라인은 표 6.3를 그대로 채우는 현장이다. 카드 거래 이벤트를 Kafka로 받고(4장), 1분 tumbling 창으로 카드 번호별 건수를 계수하고, “같은 카드, 1분 안에 3건 이상”을 임계로 설정하고, 중복 제거 키를 “카드 번호 + 창 시작 시각 + 규칙 ID”로 둔다 — 공공의 “창 시작 + 지역구 + 규칙 ID”에서 묶는 단위만 지역구에서 카드로 바뀌었다.

광고 입찰의 캠페인 예산 소진 감시는 평가 시점 선택이 공공과 반대로 기우는 경우다. 노출 이벤트를 캠페인 ID별 5분 tumbling 창으로 집계해 남은 예산 대비 소진 속도를 계산하고, 속도가 임계를 넘으면 입찰가를 낮추거나 캠페인을 중단한다. 여기서는 지연 허용을 0으로 두고 즉시 확정하는 편이 합리적이다 — 늦게 온 노출 몇 건을 빠뜨리는 손해보다 예산을 초과하는 손해가 크기 때문이다. 공공의 재난 경보가 “미확정 수치라도 빨리”를 택하는 것과 계산의 구조는 같고, 비교 대상만 인명 위험 대신 예산이다.

A/B 테스트 실시간 대시보드에서는 6.2에서 설명한 중복 집계 오류가 동일하게 발생한다. 실험군과 대조군의 전환율을 10분 sliding 창으로 비교하는데, 전환 수를 창끼리 더하면 실제의 두 배가 된다 — 감사 지적이 되는 대신 잘못된 실험 판정이 될 뿐, 오독의 구조는 같다. 대응도 같다: 확정 통계는 tumbling으로 실험 종료 후 배치(7장)에서 산출하고, 대시보드 수치에는 “잠정 추세”임을 표기한다.

네 현장을 관통하는 것은 하나다 — 창 방식과 크기, watermark 지연 허용값, output mode, 알림의 임계·평가 시점·중복 제거 키. 이 네 가지 결정은 어느 도메인에서도 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사·형평성·법적 보고 의무라면, 민간에서는 매출·비용·응답 속도다. 방향의 비대칭도 4장과 같다. 공공 설계를 민간에 도입할 때는 증거 산출 의무를 제외해도 손실이 없다 — 폐기 카운터는 수집 품질 지표가 되고, 알림 승인 기록은 오경보 원인 분석 자료가 된다.

핵심 정리

알림은 집계 위의 필터지만, 미확정 수치·중복·무입력 상태에 대응하는 절차를 함께 설계해야 한다.

빠른 알림과 정확한 알림은 트레이드오프이며, 정책을 알림별로 문서화한다.

알림 발송에도 5장의 중복 제거 키와 맥등 쓰기가 그대로 필요하다.

창 방식·watermark·output mode·알림 조건이라는 네 결정은 도메인과 무관하게 내려야 하고, 근거만 도메인에 따라 달라진다.

실습 6.1 지역별 민원 이벤트 윈도우 집계(★★★☆☆)

실습 목표

파일 소스 스트림에 watermark + tumbling window 집계를 구성하고 update 모드로 출력한다.

지연 이벤트 두 건(수용 1, 폐기 1)의 판정을 실측 증거로 확인해 지연 이벤트 처리 평가표를 만든다.

입력 데이터

민원 이벤트 14건은 시뮬레이션 입력이다(4.5장과 동일한 이유 — 개인정보가 포함된 실제 민원을 사용할 수 없다). 이 실습의 관찰 대상은 데이터 내용이 아니라 **실제 엔진의 윈도우·watermark 동작**이며, 아래 인용 수치는 모두 실제 실행 산출물에서 산출된다. 지연 이벤트의 수용·폐기 판정도 사람이 단정하지 않고, 스크립트가 집계 갱신과 폐기 카운터에서 **계산**해 평가표에 기록한다.

선수 조건과 실행

Java 17 이상이 필요하다(`JAVA_HOME` 설정). Python은 PySpark가 지원하는 버전을 사용한다(이 실습은 3.13에서 검증했다).

```
cd practice/chapter5
python3.13 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter5.py
```

스크립트는 이벤트 파일 3개를 스트림 입력 디렉터리에 순차로 “도착”시키고, 파일마다 `processAllAvailable()` 로 micro-batch 경계를 고정한다(실행마다 같은 배치 구성이 되도록). 입력 구성과 지연 이벤트 2건의 위치는 다음과 같다.

파일	이벤트(발생 시각 기준)	실험 의도
파일 1	정시 6건: 강남 22:01-22:03-22:08, 마포 22:02-22:09, 관악 22:05	창 22:00-22:10 초기 집계
파일 2	정시 4건: 강남 22:11-22:14, 마포 22:12-22:18 + LATE_OK(강남, 발생 22:04)	watermark "안" 지연 → 수용 관찰
파일 3	LATE_DROP(마포, 발생 21:49) + 정시 2건: 관악 22:21, 강남 22:24	watermark "밖" 지연 → 폐기 관찰

LATE_OK와 LATE_DROP은 이벤트 시간이 도착 순서보다 과거인 이벤트, 즉 2장에서 정의한 이벤트 시간·처리 시간의 어긋남을 의도적으로 만든 것이다. 파일 3에 정시 이벤트 2건을 함께 포함한 이유도 설계의 일부다 — 이 두 건이 watermark를 22:14까지 전진시켜, 창 22:00-22:10이 닫히는 것까지 한 실행에서 관찰된다.

핵심 코드

```
counts = (events.withWatermark("event_time", "10 minutes")
            .groupBy(F.window("event_time", "10 minutes"), "district").count())
query = counts.writeStream.outputMode("update").foreachBatch(sink).start()
query.processAllAvailable() # 파일 투입 후 배치 경계 고정
```

전체 코드는 [practice/chapter5/code/5-1-streaming-window-aggregation.py](#) 참고

실행 중 확인 체크리스트

콘솔 출력에서 다음 네 가지를 순서대로 확인하면 실습의 학습 목표를 모두 확인하게 된다.

- ☐ 파일 1 처리 후: 창 22:00-22:10에 지역구 3개 행이 나왔는가(초기 집계)
- ☐ 파일 2 처리 후: 창 22:00-22:10 강남구가 3에서 4로 **다시** 출력됐는가(지연 수용 = 과거 창 갱신)
- ☐ 파일 3 처리 후: 21:40-21:50 창이 출력에 **없고**, dropped 카운터가 1인가(지연 폐기)
- ☐ sliding 결과의 건수 합이 tumbling의 두 배인가(중첩 창의 정의)

산출물 안내

스크립트는 세 파일을 남기며, 각각 용도가 다르다.

산출물	내용	용도
ch5_window_counts.jsonl	micro-batch별 출력 행(창·지역구·건수)	실시간 집계 결과 파일 — update 출력의 원본 기록
ch5_late_event_report.json	지연 이벤트별 판정·증거, 폐기 카운터, 배치별 출력	지연 이벤트 처리 평가표 — 검수 자산
ch5_sliding_demo.json	같은 이벤트의 sliding 집계(배치 계산)	표 6.2의 tumbling-sliding 비교 근거

평가표의 판정(“수용”/“폐기”)은 스크립트가 산출물에서 계산한 값이다 — 창 건수가 기대대로 갱신되었는지, 폐기 카운터가 증가했는지를 확인해 기록하므로, 판정 자체가 검증 가능한 증거를 동반한다.

실행 결과(실제)

집계 결과는 ch5_window_counts.jsonl, 평가표는 ch5_late_event_report.json 에 저장된다.

micro-batch별 출력(바뀐 행만 — update 모드):

배치 0(파일 1 처리): 창 22:00–22:10 → 강남구 3, 마포구 2, 관악구 1
배치 2(파일 2 처리): 창 **22:00–22:10 → 강남구 4**(지연분 수용으로 갱신), 창 22:10–22:20 → 강남구 2, 마포구 2
배치 4(파일 3 처리): 창 22:20–22:30 → 강남구 1, 관악구 1 — 그리고 21:40–21:50 창은 **어디에도 없다**
배치 1·3·5: 출력 0행(파일 처리 직후 도는 빈 확인 배치 — 6.1절의 관찰)

배치 2의 출력을 해석하는 방법을 정리한다. 세 행이 나왔는데 성격이 다르다 — “22:00–22:10 강남구 4”는 **과거 창의 갱신**(지연 이벤트 수용의 결과)이고, “22:10–22:20”의 두 행은 **새 창의 첫 등장**이다. update 모드는 이 둘을 구분해 주지 않고 “바뀐 행”으로 똑같이 내보낸다. 받는 쪽에서 갱신과 신규를 구분할 필요가 없도록 만드는 것이 바로 키 기반 Upsert(5장)다 — 신규면 삽입되고 갱신이면 덮어써져, 받는 쪽 논리가 하나로 통일된다.

표 6.4 지연 이벤트 처리 평가표(실제 실행 — watermark 10분)

이벤트	발생 (이벤트 시간)	도착 시점의 watermark	속한 창	판정	증거(실측)
LATE_OK	22:04	21:59 (= 22:09 – 10분)	22:00–22:10 (끝 > watermark)	수용	강남구 창 건수 3 → 4 갱신
LATE_DROP	21:49	22:08 (= 22:18 – 10분)	21:40–21:50 (끝 < watermark)	폐기	numRowsDroppedByWatermark 0 → 1

관찰

수용의 증거와 폐기의 증거가 다른 채널에 남았다. 수용은 집계 행 갱신(3→4)으로 결과에 나타나지만, 폐기는 결과에 아무 흔적이 없고 엔진 메트릭의 카운터(1)로만 확인됐다. 폐기 모니터링 없이 운영하면 자연 데이터 손실을 인지할 방법이 없다.

update 모드는 같은 키(창 22:00~22:10, 강남구)의 행을 두 번 내보냈다(3, 그리고 4). 이 출력을 저장하는 쪽은 5장의 Upsert가 필요하다 — naive INSERT로 받으면 같은 창이 두 행이 되어, 5장 실습의 A 단계와 같은 왜곡이 재현된다.

두 자연 이벤트의 자연 폭 차이보다 중요한 것은 **watermark가 “그 이벤트가 얼마나 늦었나”가 아니라 “엔진이 최근에 무엇을 봤나”에서 계산된다는 점**이다. 스트림 전체가 함께 늦으면 (예: 수집 장애 후 몰아서 도착) watermark도 함께 늦게 전진하므로 폐기되지 않는다. 폐기는 “일부만 유독 늦을 때” 생긴다.

실행 시간의 대부분은 Spark 기동(수 초~수십 초)이다. 이벤트 14건의 처리 자체는 순간이며, 이 실습의 목적은 성능이 아니라 의미론(수용·폐기) 관찰이다.

출력 디렉터리에 checkpoint/ 가 생성된 것도 확인한다. 스트리밍 쿼리가 “어디까지 처리했고 상태가 무엇인지”를 남긴 기록으로, 표 6.1(비교표)에서 제시한 “진행 위치의 기록”의 실체다.

실습 스크립트는 재현성을 위해 매 실행 시 이를 지우고 시작하지만, 운영에서는 이 디렉터리가 장애 복구의 근거다(보충).

실행 환경에서 발생한 오류 사례

이 실습을 준비하며 실제로 발생한 오류를 기록해 둔다. 시스템 기본 python3 (3.14)와 venv의 파이썬(3.13)이 달라, 드라이버는 3.13인데 워커가 3.14로 실행되어 PYTHON_VERSION_MISMATCH 오류로 실패했다. 원인은 Spark 워커가 파이썬을 PATH에서 찾기 때문이다. 해법은 실행 인터프리터를 워커에 강제하는 것 — 실습 코드 상단의

```
os.environ.setdefault("PYSPARK_PYTHON", sys.executable)
```

이 그것이다. 분산 엔진은 “내 프로세스”와 “엔진이 띄우는 프로세스”의 환경이 다를 수 있다는 것, 그리고 이런 환경 요건(Java 17+, 파이썬 버전)은 실습 문서에 명시되어야 한다는 것을 확인하는 실제 사례다.

위험 식별(평가표 연계)

위험	실습에서의 근거	대응 방향
자연 데이터의 조용한 폐기	LATE_DROP: 결과 무흔적, 카운터로만 확인	numRowsDroppedByWatermark 상시 감시(11장)
update 출력의 저장 중복	같은 창·지역구 행이 2회 출력 (3→4)	창+지역구 키 Upsert(5장)
watermark 과소 설정	10분 넘는 지연은 폐기 대상이 됨 (엔진은 이를 “처리될 수도, 안 될 수도 있음”으로 두므로 수용을 보장하지 않는다)	실제 지연 분포 관찰 후 값 결정, 주기 재검토

보충(전문가 트랙 포인터)

상태 저장 연산의 메모리 비용: 열려 있는 모든 창×키가 엔진 상태로 유지된다. 실습은 창 3~4개×지역구 3개라 무시할 수준이지만, 운영에서 “1분 창×전국 읍면동×지연 허용 24시간”이면 상태가 수백만 항목이 된다. watermark가 상태 정리 시점을 정하므로, 지연 허용을 늘리는 것은 곧 상태를 늘리는 것이다.

체크포인트와 장애 복구: 스트리밍 쿼리는 체크포인트(오프셋·상태 스냅샷)로 재시작 시 이어서 처리한다. 실습 코드의 `checkpointLocation` 이 그것이며, 4장의 커밋된 오프셋과 같은 역할 구조다. 체크포인트를 지우고 재시작하면 소스 처음부터 다시 처리한다 — 이 재처리가 안전하려면 싱크가 5장의 맥등 쓰기여야 한다는 점에서, 세 장의 개념이 한 지점에서 연결된다.

Kafka-Spark 연계 운영 튜닝: 실습은 파일 소스였지만 운영은 대개 Kafka 소스다(커넥터 패키지 필요). 4장의 토픽·파티션 설계가 스트리밍 병렬성과 직결된다 — 파티션 수가 스트리밍 읽기 병렬성의 상한이 되므로, 표 4.2의 파티션 설계가 이 장의 처리 성능까지 규정한다.

핵심 용어

무한 테이블: 스트림을 “행이 계속 추가되는 테이블”로 보는 모델

micro-batch / trigger: 증분 실행의 단위 / 그 실행 시점의 규칙(기본 `processingTimeAvailableNow`)

이벤트 시간 / 처리 시간: 사건 발생 시각 / 시스템 도착 시각(2장) — 창 집계 기준은 전자

output mode(`append/update/complete`): 확정 행만 / 바뀐 행만 / 전체

tumbling / sliding window: 비중첩 창 / 중첩 창(한 이벤트가 여러 창에)

watermark: 관측된 최대 이벤트 시각 – 지연 허용. 창 닫힘과 폐기의 기준선

numRowsDroppedByWatermark: watermark 폐기 건수 메트릭 — 집계 출력만 볼 때 폐기의 유일한 직접 증거(운영에서는 원천 입력 로그·감사 로그와 대조해 보완할 수 있다)

상태(state): 열려 있는 창들의 중간 집계 — 증분 계산의 근거이자 메모리 비용의 원천

체크포인트: 쿼리의 진행 위치·상태 스냅샷 — 4장 커밋 오프셋의 스트리밍판

다음 장 예고

이 장의 스트리밍 집계는 “도착하는 대로”의 세계였다. 그러나 6.3에서 확인했듯 스트리밍의 수치는 watermark 안에서만 지연을 수용한 근사 확정치다 — 폐기된 이벤트, watermark 너머의 지연분은 반영되지 않는다. 7장은 그 반대편, 축적된 데이터를 정기적으로 다시 가공하는 배치 분석을 다룬다. 스트리밍이 만든 빠른 수치와 배치가 만든 확정 통계가 어떻게 역할을 나누고 서로를 검증하는지 (3장 하이브리드 설계의 실체), 그리고 배치 실패의 재실행이 왜 또다시 5장의 맥등성을 요구하는지 다룬다.

이 장의 스트리밍 경로가 최종적으로 어디서 검증되는지도 미리 가리켜 둔다. 14장은 이 책의 개념을 하나의 통합 시스템으로 재조립하면서, **스트림 경로로 집계한 확정치가 7장의 배치 확정치와 일치하는지**를 수치 대조로 증명한다(사흘치 마포·관악·강남 5·5·8 / 5·5·6 / 6·5·9 — 배치와 스트림이 완전히 다른 경로인데 같은 값). 람다 아키텍처(3장)의 대표적 문제인 “두 경로의 결과 불일치”를 검증 가능한 형태로 확인하는 것이다. 그리고 이 장이 다룬 지연 도착의 종착점도 14장에 있다 — 지연 레이블로 매긴 예측 채점 6건 전체의 평균절대오차(MAE)가 1.0으로 기록된다. 레이블이 하루 늦게 도착하는 지연 평가의 실체이며, 6장에서 watermark가 지연 이벤트의 수용 또는 폐기를 판정했던 방식이 하루 평가 계층에서 어떻게 다시 나타나는지 확인한다.

참고문헌

Apache Spark. (n.d.). *Structured Streaming Programming Guide*.

<https://spark.apache.org/docs/latest/streaming/index.html>

Apache Spark. (n.d.). *Structured Streaming Programming Guide (단일 페이지)*.

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Apache Spark. (n.d.). *PySpark Structured Streaming API Reference*.

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.ss/index.html>

Akidau, T., Bradshaw, R., Chambers, C., Chernyak, S., Fernández-Moctezuma, R. J., Lax, R., McVeety, S., Mills, D., Perry, F., Schmidt, E., & Whittle, S. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12), 1792–1803. <https://www.vldb.org/pvldb/vol8/p1792-Akidau.pdf> — 방법론·개념 차용 (스트리밍 일반 대상, 공공 직접 선례 아님)

Apache Flink. (n.d.). *Timely Stream Processing — Event Time and Watermarks*.

<https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/> — 배경·타 엔진 대조

Apache Flink. (n.d.). *Generating Watermarks*. https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/event-time/generating_watermarks/ — 배경·타 엔진 대조

Databricks. (2022). *Feature Deep Dive: Watermarking in Apache Spark Structured Streaming*.

<https://www.databricks.com/blog/2022/08/22/feature-deep-dive-watermarking-apache-spark-structured-streaming.html> — 현장 방법 차용(벤더 실무 가이드)